

Diseño y Documentación de Arquitectura de Sistemas IA

Evaluación Parcial N°1 — Informe Técnico

Sistema de Recomendación de Productos para E-commerce (ShopFast)

- **Asignatura:** Arquitectura de Sistemas de Inteligencia Artificial
- **Sigla:** ITY1102
- **Institución:** Duoc UC
- **Tipo de evaluación:** Parcial 1
- **Plantilla:** arc42 simplificada
- **Caso empresarial:** Caso 1 — Sistema de Recomendación para E-commerce (ShopFast)

Equipo N°: 1

Integrantes:

- Martin Higuera
- Gabriel Durán
- Francisco Salazar

Docente: Ricardo Aravena Videla

Fecha de entrega: Semana 7 — 24 / 09 / 2026

Índice

- **1.1 Análisis del caso empresarial**
 - 1.1.1 Contexto del caso y problemática
 - 1.1.2 Objetivo del proyecto
 - 1.1.3 Requerimientos funcionales
 - 1.1.4 Requerimientos no funcionales
 - 1.1.5 Componentes principales de la arquitectura
 - 1.1.6 Ciclo de vida y elementos de los datos
 - 1.1.7 Elementos AWS y su función
- **1.2 Principios de diseño arquitectónico**
 - 1.2.1 Escalabilidad
 - 1.2.2 Flexibilidad
 - 1.2.3 Seguridad
 - 1.2.4 Observabilidad
 - 1.2.5 Confiabilidad
 - 1.2.6 Decisiones arquitectónicas clave
- **1.3 Especificación de infraestructura**
 - 1.3.1 Recursos computacionales (CPU, GPU y TPU)
 - 1.3.2 Selección de frameworks de desarrollo de IA
 - 1.3.3 Selección de servicios cloud (AWS, Azure, GCP)
 - 1.3.4 Dimensionamiento inicial con estimación de costos
- **1.4 Estrategias de integración y despliegue**
 - 1.4.1 Uso de edge computing
 - 1.4.2 Comparación de alternativas de integración
 - 1.4.3 Pipeline de integración continua y despliegue continuo (CI/CD)
 - 1.4.4 Diagrama de flujo de despliegue
- **Referencias**
- **Declaración de uso de herramientas de IA generativa**

1.1 Análisis del caso empresarial

1.1.1 Contexto del caso y problemática

ShopFast presenta estancamiento de ventas en los últimos 6 meses: los usuarios no encuentran productos interesantes, lo que origina una conversión del 2.3% y un valor promedio de compra de 85 USD.

1.1.2 Objetivo del proyecto

Se busca un sistema de recomendación que eleve la conversión a 4% y el ticket promedio a 120 USD mediante sugerencias personalizadas.

1.1.3 Requerimientos funcionales

R.F 1: El sistema debe permitir al usuario buscar y filtrar productos del catálogo a partir de distintos atributos como nombre, categoría y rango de precios.

R.F 2: El sistema debe permitir al usuario visualizar la información detallada de un producto seleccionado, incluyendo su imagen, nombre, descripción, especificaciones y precio.

R.F 3: El sistema debe mostrar en la página de inicio (Homepage) una sección personalizada denominada "Recomendado para ti" con un listado de hasta 10 productos sugeridos según el perfil e historial del usuario.

R.F 4: El sistema debe mostrar en la página de detalle de cada producto una sección denominada "También te puede interesar" con un listado de hasta 5 productos complementarios o alternativos.

R.F 5: El sistema debe generar recomendaciones personalizadas para cada usuario activo a partir del análisis de su comportamiento y similitud de patrones con otros usuarios.

R.F 6: El sistema debe registrar en tiempo real las interacciones de los usuarios, tales como clics en productos, visualizaciones de páginas, adiciones al carrito y compras realizadas.

R.F 7: El sistema debe permitir registrar la valoración de los usuarios respecto a los productos, ya sea de forma explícita o deducida a partir de su historial de compras e interacciones.

R.F 8: El sistema debe retornar un listado de los productos más vendidos como alternativa por defecto (fallback) cuando el usuario sea nuevo, no cuente con historial previo o el servicio de recomendaciones presente demoras o fallos.

R.F 9: El sistema debe permitir a las aplicaciones cliente solicitar sugerencias personalizadas mediante un identificador de usuario y recibir una lista de productos ordenados por un puntaje de relevancia.

R.F 10: El sistema debe permitir la interacción directa con las tarjetas de productos recomendados para ver su detalle o añadirlos al carrito, contabilizando la acción para la medición del porcentaje de clics (CTR).

R.F 11: El sistema debe exponer un mecanismo para consultar y verificar el estado de salud y disponibilidad operativa del servicio de recomendaciones.

1.1.4 Requerimientos no funcionales

R.N.F 1: El sistema debe responder a las solicitudes de recomendaciones con un tiempo de latencia menor a 500 milisegundos en peticiones cálidas (*warm requests*).

R.N.F 2: El servicio de recomendaciones debe mantener una disponibilidad operativa mínima del 99% durante el horario comercial.

R.N.F 3: El sistema debe soportar una capacidad de procesamiento concurrente de al menos 500 usuarios simultáneos y un tráfico pico de 2.100 peticiones por hora sin degradación del servicio.

R.N.F 4: La infraestructura computacional en la nube (AWS) debe operar bajo un esquema de costos optimizado que no supere los \$200 USD mensuales.

R.N.F 5: La arquitectura debe ser serverless y desacoplada mediante AWS Lambda y API Gateway, permitiendo un autoescalado elástico de 0 a 1.000 instancias según la demanda.

R.N.F 6: La persistencia de datos relacionales debe gestionarse a través de Amazon RDS con motor MySQL, complementada con Amazon S3 para almacenamiento duradero de modelos y datos históricos.

R.N.F 7: El motor de recomendación debe implementarse mediante técnicas de filtrado colaborativo con Factorización de Matrices (SVD), logrando un error cuadrático medio (RMSE) igual o menor a 0.85 y una precisión@10 de al menos 0.72.

R.N.F 8: Todas las comunicaciones y transferencias de datos entre el cliente web y las APIs deben estar cifradas mediante el protocolo seguro HTTPS.

R.N.F 9: El acceso a los endpoints de la API debe estar autenticado mediante API Keys gestionadas de forma segura en AWS Systems Manager Parameter Store y con políticas de acceso IAM basadas en el principio de mínimo privilegio.

R.N.F 10: La API debe implementar un control de tasa de consumo (*rate limiting*) configurado a un máximo de 100 solicitudes por minuto por cliente.

R.N.F 11: Los registros y trazas del sistema no deben contener información de identificación personal (PII) y deben incluir un identificador único por petición (*request_id*) para asegurar la observabilidad en Amazon CloudWatch.

R.N.F 12: El sistema debe contar con un mecanismo de resiliencia que active un reintento automático y un tiempo límite de espera (timeout) de 2 segundos, disparando el fallback de productos en menos de 500 milisegundos ante fallos de backend.

R.N.F 13: El proceso de integración y despliegue continuo (CI/CD) debe ejecutar despliegues de producción progresivos tipo Canary (10% por minuto) con reversión automática (*rollback*) si la tasa de errores supera el 3%.

R.N.F 14: La arquitectura debe ser flexible y desacoplada, permitiendo la sustitución o actualización del algoritmo del modelo de aprendizaje automático sin interrumpir la operación continua del servicio.

1.1.5 Componentes principales de la arquitectura

Los cuatro componentes exigidos se materializan en ShopFast así:

Componente	Función específica	Tecnología en el caso (ShopFast)	Requerimiento(s) que satisface
Datos	Ingesta, limpieza y persistencia del histórico de interacciones y del catálogo que alimentan al motor	Amazon RDS (MySQL) para transacciones en caliente; Amazon S3 como data lake (12 meses, 2 GB); pandas para transformación	R.F 6, R.F 7, R.N.F 6
Modelo	Generar recomendaciones personalizadas: filtrado colaborativo con puntaje de relevancia y alternativa de <i>fallback</i>	Python Surprise 1.1.3 (SVD); entrenamiento en Amazon SageMaker (ml.t3.medium, semanal) y artefacto serializado versionado en S3 (Model Registry)	R.F 5, R.F 8, R.N.F 7, R.N.F 14
API	Exponer el modelo como servicio REST seguro, disponible y con control de consumo	Flask sobre AWS Lambda + API Gateway: GET /recommendations/{user_id}, POST /track-interaction, GET /health	R.F 9, R.F 11, R.N.F 1, R.N.F 5, R.N.F 9, R.N.F 10, R.N.F 12
Interfaz	Presentar las recomendaciones al usuario final y permitir su interacción	Frontend React: homepage "Recomendado para ti" (10 productos) y página de producto "También te puede interesar" (5 productos), con tarjetas de imagen, nombre, precio y botón	R.F 1, R.F 2, R.F 3, R.F 4, R.F 10

Relacionando componentes y funciones mediante modelos de referencia (apartado c de la pauta), los cuatro encadenan el ciclo **MLOps**: *ingesta* (RDS MySQL + S3) → *entrenamiento* (SageMaker, disparado por EventBridge) → *registro* (S3 como Model Registry del SVD) → *despliegue* (API Gateway + Lambda) → *monitoreo* (CloudWatch y POST /track-interaction). Es la manifestación de la **arquitectura de referencia de AWS** para workloads de IA, donde cada servicio cubre una etapa sin acoplar el modelo al código, cumpliendo R.N.F 14.

1.1.6 Ciclo de vida y elementos de los datos

El ciclo de vida abarca la ingesta, el procesamiento, el almacenamiento y el consumo por el modelo.

Ingesta de los datos

Se alimenta de dos fuentes: la base de datos AWS de motor MySQL y los archivos históricos de S3 con hasta 12 meses de antigüedad; ambos pasan por un pipeline que asegura datos limpios.

Entidades y atributos clave

Las *features* son **user_id**, **product_id**, **interaction_type**, **rating**, **timestamp**, **categoría**, **precio**; su rol en el modelo es el siguiente:

Atributo (<i>feature</i>)	Descripción y rol en el modelo
user_id	Identifica de forma única al usuario; permite asociar y personalizar los productos recomendados para cada cliente.
product_id	Identifica de forma único el producto a recomendar.
interaction_type	Registra el tipo de interacción (clic, vista, compra); permite al modelo ponderar los comportamientos (p. ej., recomendar productos similares a los comprados).
rating	Calificación del usuario; permite discriminar productos según su recepción por parte de los usuarios.
timestamp	Guarda la fecha y hora de la interacción; sirve para auditoría y análisis temporal.
categoría	Ayuda a separar los productos en clases (categorías), facilitando el filtrado y la organización del catálogo.
precio	Guarda el precio de un producto; permite filtrar recomendaciones según el presupuesto del usuario.

Procesamiento y transformación de los datos

Los datos se transforman en un pipeline con **pandas**, limpiando valores nulos y atípicos que rompan reglas de negocio. Se preserva la integridad con copias de respaldo y la eliminación de datos sensibles; los datasets se entregan en formato CSV.

Almacenamiento de los datos

Cada tipo de dato reposa según su uso: Amazon RDS (MySQL) para las transacciones diarias y Amazon S3 como *data lake* para el histórico y los artefactos del modelo. Se emplea un data lake porque la fuente permanece sin procesar y su costo es menor.

1.1.7 Elementos AWS y su función

Elementos de AWS y su función en el proyecto:

- **Amazon SageMaker:** cómputo de entrenamiento; ejecuta el job semanal en ml.t3.medium, entrena 150K interacciones en 15 minutos y se apaga.
- **Amazon EventBridge:** programador de tareas que dispara el reentrenamiento semanal, cada domingo a las 2:00 AM.
- **Amazon S3 (Simple Storage Service):** data lake y Model Registry; guarda el historial de 12 meses y el modelo SVD serializado.
- **AWS Lambda:** cómputo serverless de inferencia; resuelve `/recommendations/{user_id}` en milisegundos, sin servidores dedicados.
- **Amazon API Gateway:** gestión de APIs; enruta HTTP hacia Lambda, cifra en HTTPS y aplica autenticación por API Key y *rate limiting* de 100 req/min.
- **Amazon RDS (MySQL):** base de datos transaccional (*OLTP*) de las interacciones diarias en caliente (clics, compras, catálogo).
- **Amazon CloudWatch:** observabilidad; recolecta logs con `request_id`, genera métricas de latencia (p50, p95, p99) y alarma sobre el umbral crítico.
- **AWS Systems Manager Parameter Store:** seguridad y configuración; guarda credenciales y API Keys desacopladas del código fuente.

1.2 Principios de diseño arquitectónico

Principios arquitectónicos que el proyecto cumple: escalabilidad, flexibilidad, seguridad, observabilidad y confiabilidad.

1.2.1 Escalabilidad

La escalabilidad es horizontal, automática y elástica: AWS Lambda y Amazon API Gateway escalan de 0 a 1.000 instancias concurrentes (validado con 500 usuarios sin errores). En Machine Learning, SageMaker y EventBridge provisionan cómputo efímero solo durante los 15 minutos del reentrenamiento semanal (\$0.20/mes en entrenamiento y \$28/mes en inferencia). El balanceo redirige el tráfico ante fallos y reduce instancias ante menor demanda.

1.2.2 Flexibilidad

La flexibilidad se logra desacoplando la lógica de servicio de los artefactos de Machine Learning: el modelo vive como archivo binario en Amazon S3 y las credenciales en AWS Systems Manager Parameter Store. Así se actualiza el motor sin tocar el código de la API ni los contratos REST de AWS Lambda y API Gateway; validado migrando de SVD a Alternating Least Squares (ALS) en 2 horas sin interrumpir el frontend.

1.2.3 Seguridad

La seguridad aplica defensa en profundidad y mínimo privilegio con AWS IAM: cada componente dispone solo de los permisos necesarios. En el perímetro, Amazon API Gateway impone cifrado HTTPS, API Keys y *rate limiting* a 100 req/min; las credenciales se cifran en AWS Systems Manager Parameter Store, sin secretos en el código. Los datasets se anonimizan y los logs de Amazon CloudWatch carecen de PII, con cero vulnerabilidades críticas en pruebas de penetración (*penetration testing*).

1.2.4 Observabilidad

La observabilidad se garantiza con Amazon CloudWatch (Logs, Metrics, Dashboards y Alarms): las trazas de AWS Lambda se emiten en JSON con `request_id` y sin PII. El monitoreo supervisa latencia (p50 235 ms, p95 420 ms, p99 780 ms), arranques en frío (4.2%) y errores (0.8%), con alarmas Warning (>800 ms) y Crítica (>1.5 s o >5%); permite detectar incidentes en menos de 15 minutos y gobierna los rollbacks del pipeline.

1.2.5 Confiabilidad

El sistema aplica tolerancia a fallos y degradación elegante (*graceful degradation*) con una disponibilidad del 99.4%: reintento ante errores 5xx, timeout de 2 segundos y fallback de productos más vendidos cacheados en menos de 500 ms. La confiabilidad de los despliegues se asegura con CI/CD (GitHub Actions y AWS SAM) en Canary al 10% por minuto, con rollback si el error supera el 3%; el modelo inmutable en S3 y el endpoint GET /health garantizan recuperabilidad y verificación del estado.

1.2.6 Decisiones arquitectónicas clave, basadas en principios de observabilidad y confiabilidad

Decisión 1 — Despliegue Canary gobernado por alarmas de CloudWatch - Decisión: GitHub Actions + AWS SAM despliegan el 10% de tráfico por minuto; CloudWatch revierte si el error supera el 3%. - **Principio(s):** observabilidad y confiabilidad. - **Alternativas descartadas:** *big bang* (100% del tráfico sin señal previa) y *blue/green* completo (duplica infraestructura y el costo del R.N.F 4). - **Consecuencia/validación:** cumple R.N.F 13 y sostiene un uptime de 99.4% sobre el 99% exigido en R.N.F 2.

Decisión 2 — Resiliencia en inferencia: retry, timeout 2 s y fallback - Decisión: ante errores 5xx el cliente reintenta con timeout de 2 s y, si el motor falla, degrada a productos más vendidos cacheados. - **Principio(s):** confiabilidad. - **Alternativas descartadas:** propagar el error al usuario y reintentos superiores a 2 s (incompatible con la p95 de 420 ms). - **Consecuencia/validación:** cumple R.N.F 12 (fallback <500 ms) y R.F 8, con p50 de 235 ms y error de 0.8%.

Decisión 3 — Trazas request_id sin PII y alarmas segmentadas - Decisión: Lambda emite logs JSON con request_id y sin PII; CloudWatch define alarmas Warning (>800 ms) y Crítica (>1.5 s o >5%). - **Principio(s):** observabilidad. - **Alternativas descartadas:** logs en texto libre (impiden correlación) y un solo umbral (fatiga o detección tardía). - **Consecuencia/validación:** cumple R.N.F 11 y detecta incidentes en menos de 15 minutos con p50 235 ms, p95 420 ms, p99 780 ms, cold starts 4.2% y errores 0.8%.

Decisión 4 — Modelo inmutable en S3 y health check /health - Decisión: cada modelo SVD se registra como artefacto inmutable y versionado en S3 y API Gateway expone GET /health. - **Principio(s):** confiabilidad y observabilidad. - **Alternativas descartadas:** sobrescribir el modelo (impide revertir) y confiar en la ausencia de errores (detección reactiva). - **Consecuencia/validación:** garantiza la recuperabilidad del artefacto y la actualización sin interrupción del R.N.F 14, coherente con R.N.F 6.

1.3 Especificación de infraestructura

Los recursos computacionales se separan en dos apartados: **entrenamiento e inferencia en tiempo real.**

1.3.1 Recursos computacionales (CPU, GPU y TPU)

Fase de entrenamiento (Amazon SageMaker):

- **Recursos asignados:** instancia **ml.t3.medium** con **2 vCPU y 4 GB** de RAM.
- **Evaluación de GPU/TPU:** no se requieren aceleradores: el SVD de Surprise es CPU-intensivo y no aprovecha paralelismo de tensores; con este perfil, los 150.000 registros entrenan en 15 minutos a \$0.20 USD/mes.

Fase de inferencia en tiempo real (AWS Lambda):

- **Recursos asignados:** funciones serverless con **1 GB de memoria RAM**.
- **Evaluación de GPU/TPU:** no requiere aceleración dedicada: la CPU serverless procesa solicitudes en 300 ms (*warm requests*) y escala de 0 a 1.000 instancias.

1.3.2 Selección de frameworks de desarrollo de IA apropiados

Framework de IA seleccionado:

- **Python Surprise (versión 1.1.3):** recomendación con filtrado colaborativo y factorización matricial (SVD/ALS), optimizada en CPU sin tensores ni GPU; entrena 150.000 interacciones en 15 minutos con RMSE 0.85 y Precisión@10 de 0.72.

Librerías complementarias:

- **Pandas:** extracción, limpieza y estructuración tabular de las variables (user_id, product_id, rating, timestamp) antes de alimentar el algoritmo.
- **Flask / Werkzeug:** microframework que encapsula el artefacto serializado y lo sirve como **API REST** dentro del contenedor serverless de AWS Lambda, con latencia p50 de 235 ms.

Cuestionamiento de frameworks o librerías como TensorFlow / PyTorch:

- Aunque TensorFlow (con TensorFlow Recommenders) o PyTorch son estándar para redes profundas (TwoTower, autoencoders), para los 5.000 productos y 50.000 usuarios actuales, **Surprise ofrece menor complejidad de despliegue, empaquetado mínimo para no penalizar el cold start de Lambda y**

cumplimiento del presupuesto (\$0.20 USD en entrenamiento). Queda reservada como evolución si el catálogo escala o requiere *embeddings* multimodales.

1.3.3 Selección de servicios cloud (AWS, Azure, GCP) considerando almacenamiento, redes de comunicación y escalabilidad

La solución se despliega sobre Amazon Web Services (AWS) bajo un patrón puramente *serverless* y desacoplado, con \$50 USD/mes frente al límite de \$200 USD/mes:

- **Almacenamiento:** Amazon RDS (MySQL) para las transacciones de interacciones y catálogo, más Amazon S3 para 2 GB de datos históricos y versionar artefactos del modelo.
- **Redes y Comunicación:** Amazon API Gateway centraliza el enrutamiento HTTP, el cifrado HTTPS, el *rate limiting* (100 req/min) y la autenticación con API Keys en AWS Systems Manager Parameter Store; Amazon EventBridge orquesta los eventos temporales.
- **Cómputo y Escalabilidad:** cómputo elástico horizontal con AWS Lambda (0 a 1.000 instancias, latencia warm de 300 ms) y cómputo efímero vertical con Amazon SageMaker (ml.t3.medium que entrena 150K registros en 15 minutos y se apaga).
- **Observabilidad:** Amazon CloudWatch gobierna la salud de red y cómputo con logs estructurados, dashboards y alertas.

1.3.4 Dimensionamiento inicial de recursos con estimación de costos

El proyecto tiene un tope de inversión de **200\$ USD por mes**; su desglose es el siguiente:

Componente / Servicio	Dimensionamiento de recursos	Frecuencia / Volumen de uso	Costo estimado mensual
Entrenamiento (Amazon SageMaker)	Instancia ml.t3.medium (2 vCPU, 4 GB RAM)	Semanal (domingos 2 AM), 15 minutos de ejecución por corrida	\$0.20 USD/mes
Inferencia (AWS Lambda)	Función serverless con 1 GB de memoria RAM	~60.000 invocaciones al mes bajo demanda	\$28.00 USD/mes
Almacenamiento en la nube (Amazon S3)	2 GB de datos históricos y artefactos de modelos serializados	Almacenamiento continuo persistente	\$0.05 USD/mes
Base de datos relacional (Amazon RDS)	db.t3.micro con MySQL y 20 GB de almacenamiento	24/7 para operaciones transaccionales	\$16.00 USD/mes
Costo total de la solución	Infraestructura serverless + RDS	—	~\$50.00 USD/mes (aprox. \$44.25 – \$50 USD)

Costo de arquitectura tradicional (alternativa descartada): instancias tipo Amazon EC2 24/7 costarían **\$180 USD/mes**, cifra cara para el presupuesto, mientras que la solución serverless (Lambda + SageMaker) factura solo los milisegundos de cómputo de las invocaciones y los 15 minutos de entrenamiento semanal.

1.4 Estrategias de integración y despliegue

Se seleccionó **CI/CD (Continuous Integration / Continuous Deployment)** como estrategia de integración y entrega: **GitHub Actions** filtra la calidad antes de producción, controlando fallos, y **AWS SAM** complementa ese control en el despliegue.

Como despliegue de modelos en producción se eligió **Canary Deployment**, que minimiza el riesgo al progresar el despliegue a medida que las métricas son positivas.

1.4.1 Uso de edge computing

Dentro de las estrategias de despliegue se evaluó el *edge computing*: ejecutar lógica y almacenar contenido cerca del usuario. Para ShopFast se recomienda una **adopción parcial** coherente con su patrón *serverless*: servir el frontend React con **Amazon CloudFront** y usar **Lambda@Edge** para cachear el *fallback* de más vendidos (R.F 8), cumpliendo menos de 500 ms aunque el motor falle (R.N.F 12) y sin duplicar infraestructura (~\$50 de un tope de \$200/mes). Se descarta la inferencia SVD en *edge*: el modelo se reentrena de forma centralizada y semanal (SageMaker) con estado en RDS y S3, y serializarlo en nodos distribuidos encarecería el sistema sin superar la latencia *warm* de 300 ms. El cómputo *edge* queda limitado a entrega y caché, no a inferencia.

1.4.2 Comparación de alternativas de integración evaluando eficiencia, rendimiento y confiabilidad

Se evaluaron además las estrategias **Blue/Green Deployment**, **Rolling (Progresivo)**, **Big Bang**, **Despliegue por fases** y **Despliegue Shadow**:

Estrategia	Eficiencia	Rendimiento	Confiabilidad	Decisión en este caso
Blue/Green	Baja: duplica entornos y costos	Alto: conmutación instantánea de tráfico	Alta: reversión inmediata	Descartada: duplicaría el presupuesto de \$200/mes
Rolling (Progresivo)	Alta: reutiliza una sola infraestructura	Medio: depende de compatibilidad entre versiones	Media: reversión más lenta	Descartada: poca visión del rendimiento del modelo en producción
Big Bang	Alta: un solo evento de despliegue	Riesgoso: pico único sin validación previa	Baja: sin reversión gradual	Descartada: máxima exposición de los 50.000 clientes a fallos
Por fases	Media: exige segmentar por grupos o regiones	Alto dentro de cada segmento	Media-alta: daño acotado por segmento	Descartada: mayor costo de soporte e inversión por regiones
Shadow	Media: duplica tráfico sin impacto en usuario	Alto para comparar versiones en paralelo	Alta para validación, nula para el usuario	Descartada: el usuario nunca ve el modelo, no mide la conversión
Canary (elegida)	Alta: un solo entorno, despliegue gradual de 10% por minuto	Se valida con tráfico real desde el primer minuto	Alta: rollback automático si el error rate supera el 3%	Elegida: progresión hasta el 100% con reversión automática

Se elige Canary por ser la más **eficiente** (un solo entorno *serverless*, sin duplicar costos), la de mejor **rendimiento** (modelo SVD medido con tráfico real desde el 10%) y la más **confiable** (CloudWatch con rollback ante errores >3%). Las alternativas son:

Blue/Green Deployment

Mantiene dos entornos de producción idénticos: uno activo con el 100% del tráfico (**Blue**) y otro inactivo con la nueva versión (**Green**); validado Green, el enrutador cambia todo el tráfico de golpe. Ofrece reversiones inmediatas y cero inactividad, pero el **principal problema es el costo**: duplica entornos y presupuesto, lo que la hace financieramente inviable.

Rolling (Progresivo)

Despliegue gradual que sustituye versiones instancia por instancia, de forma escalonada. Aunque es eficaz para microservicios, exige alta compatibilidad entre códigos y bases de datos; en funciones complejas, como integrar pipelines y modelos de aprendizaje continuo, cuesta tiempo y dinero, porque no permite ver gradualmente el rendimiento del modelo en producción.

Big Bang

Apaga el sistema antiguo y activa el nuevo de golpe para el 100% de los usuarios, en un solo evento. Es el más riesgoso y no permite calificar gradualmente el rendimiento del modelo.

Despliegue por fases

Se segmenta por grupos de usuarios, regiones geográficas, áreas de la empresa o módulos. En un comercio electrónico es clave que la mayor cantidad de usuarios compre y reciba recomendaciones; además, desplegar por regiones implica mayor inversión, especialmente en soporte.

Despliegue Shadow

Duplica el tráfico de la versión activa y envía una copia a una versión oculta (*shadow*) que procesa en segundo plano; sus respuestas se registran para análisis, pero no llegan al usuario. El proyecto busca llevar la conversión al 4% y el ticket a \$120 USD, lo que obliga a que los usuarios reales vean las recomendaciones y decidan si hacen clic, algo que esta estrategia no ofrece.

1.4.3 Pipeline de integración continua y despliegue continuo (CI/CD)

ShopFast implementa un flujo de CI/CD automatizado para minimizar errores en producción, asegurar calidad y mitigar riesgos mediante liberaciones progresivas.

Herramientas involucradas y su función

- **GitHub Actions**: orquestación central; escucha los eventos de código (*push* a main) y ejecuta los flujos (*workflows*) de compilación, análisis y despliegue.
- **pytest**: pruebas automatizadas en Python; corre unitarias y de integración en la construcción y en *staging*.
- **AWS SAM (Serverless Application Model)**: IaC serverless; empaqueta la función AWS Lambda, define los contratos de Amazon API Gateway y pondera el tráfico en los despliegues.
- **Amazon CloudWatch**: monitoreo en tiempo real; supervisa el error en la fase canary para activar la reversión automática.

Flujo del pipeline paso a paso

El proceso se ejecuta en 5 etapas secuenciales:

1. **Push a rama main y construcción (Build):** - El desarrollador sube los cambios aprobados al repositorio. - Se ejecutan pruebas unitarias (pytest), análisis estático (*linting*) y escaneos de seguridad.
2. **Despliegue a staging y pruebas de integración:** - Si es exitoso, AWS SAM despliega los artefactos en un entorno *staging* idéntico a producción. - 5 pruebas de integración automáticas certifican la comunicación entre endpoints y bases de datos.
3. **Aprobación manual (*gate de control*):** - Un responsable técnico valida *staging* y aprueba el paso a producción.
4. **Despliegue progresivo en producción (Canary):** - AWS SAM enruta inicialmente solo el 10% del tráfico real a la nueva versión. - Aumenta un 10% por minuto hasta el 100%, si las condiciones son estables.
5. **Monitoreo y reversión automática (*rollback*):** - Se evalúan en CloudWatch las métricas durante el despliegue canary. - Si el error rate supera el 3%, el pipeline interrumpe y ejecuta un rollback instantáneo hacia la versión previa estable.

1.4.4 Diagrama de flujo de despliegue

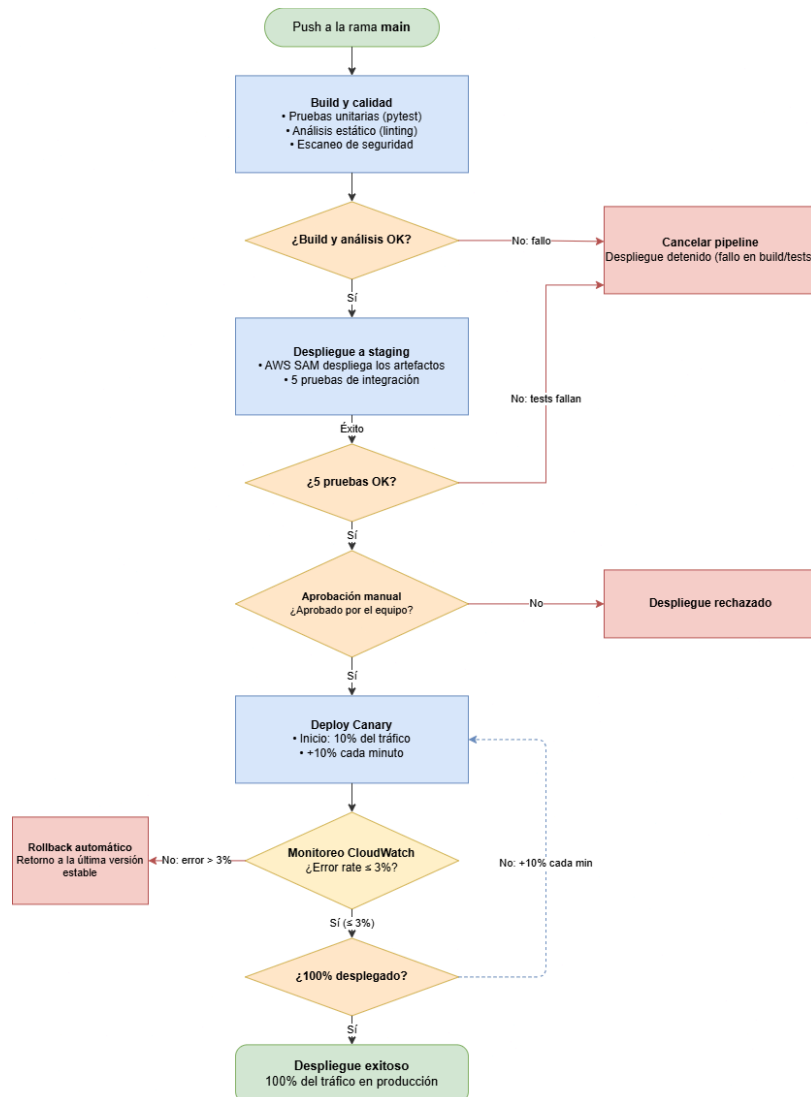


Diagrama elaborado en Draw.io (fuente editable: DiagramaShopPipeline_v2.drawio).

Referencias

- Amazon Web Services. (2025, 19 de noviembre). *Machine Learning Lens — AWS Well-Architected Framework*. Recuperado el 22 de septiembre de 2026, de <https://docs.aws.amazon.com/wellarchitected/latest/machine-learning-lens/machine-learning-lens.html>
- Kreuzberger, D., Kühl, N., & Hirschl, S. (2023). Machine Learning Operations (MLOps): Overview, Definition, and Architecture. *IEEE Access*, 11, 31866–31879. <https://doi.org/10.1109/ACCESS.2023.3262138>
- Testi, M., Ballabio, M., Frontoni, E., Iannello, G., Moccia, S., Soda, P., & Vessio, G. (2022). MLOps: A taxonomy and a methodology. *IEEE Access*, 10, 63606–63618. <https://doi.org/10.1109/ACCESS.2022.3181730>

Declaración de uso de herramientas de IA generativa

Durante el desarrollo de este trabajo se utilizaron herramientas de inteligencia artificial generativa como apoyo al proceso de investigación, redacción y revisión. El uso de estas herramientas se resume a continuación, indicando la herramienta empleada y el propósito específico en cada caso:

Herramienta	Propósito de uso
OpenCode Zen	Redacción inicial, reestructuración y revisión de coherencia y estilo de las secciones del informe.
OpenCode Zen	Generación de borradores de código (plantillas AWS SAM, flujos de GitHub Actions) y bocetos de diagramas de arquitectura, posteriormente editados por el equipo.
Documentación técnica oficial (AWS, GitHub) y verificación manual	Todo el contenido técnico —arquitectura, servicios, estrategia de despliegue canary y referencias— fue contrastado y verificado por el equipo contra la documentación oficial antes de su inclusión; la responsabilidad del contenido final es del equipo.